Reporte Técnico de Actividades Práctico-Experimentales Nro. 05

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante	Cael Alejandro Soto Castillo
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad..
Práctica Nro.	05
Título de la Práctica	Implementación y análisis del Generador Congruencial Lineal en Python y Java
Nombre del Docente	José O. Guamán Q.
Fecha	29 de octubre de 2026 30 de octubre de 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica

Comprender el funcionamiento de un generador pseudoaleatorio mediante la implementación del método LCG.

1. Implementar un generador LCG en Python.
2. Implementar el mismo algoritmo en Java.
3. Comparar las secuencias generadas y analizar sus propiedades.

3. Materiales y reactivos

- Computador.
- Python.
- Java/JDK.

4. Equipos y herramientas

- Python.
- JDK.
- Sistema operativo Linux o macOS.

5. Procedimiento / Metodología Ejecutada

5.1. Fundamento teórico

Según lo entendido en la ficha, un generador congruencial lineal produce números que parecen aleatorios usando una fórmula, además, se parte de un valor inicial que le conocemos como **semilla** y cada número nuevo se calcula a partir del anterior [1]:

$$X_{n+1} = (a \cdot X_n + c) \bmod m, \qquad U_n = \frac{X_n}{m}.$$

Aquí a es el multiplicador, c el incremento, m el módulo y X_0 la semilla.

Como al dividir entre m el resultado queda entre 0 y 1, los U_n se pueden usar como números uniformes en el intervalo $[0, 1)$.

También se puede decir que la calidad del generador depende de los parámetros y de la semilla seleccionada. Los valores que pide la ficha son:

Tabla 1: Configuración común a Python y Java. Los valores proceden de la ficha; la segunda semilla la elegí yo.

Parámetro	Valor
Multiplicador a	1664525
Incremento c	1013904223
Módulo m	$2^{32} = 4\,294\,967\,296$
Semilla X_0	12345
Cantidad de valores generados	10 000
Segunda semilla (paso “cambiar la semilla”)	54321

El **período** es la cantidad de valores que se generan antes de que la secuencia empiece a repetirse; el máximo posible es m.

5.2. Reglas que seguí en los dos lenguajes

Para que Python y Java produzcan exactamente lo mismo, definí estas reglas antes de programar:

- **Semilla.** La semilla es X_0 y no se cuenta entre los valores generados. El primer valor es X_1 y el último es X_{10000}.
- **Solo enteros.** X_n se calcula únicamente con aritmética de enteros, con la fórmula de la guía y sin redondeos. U_n se obtiene al final, dividiendo entre m.
- **Mismo formato de salida.** Cada lenguaje guarda su secuencia en un CSV con las columnas `n`, `x` y `u` (esta última con 10 decimales).
- **Comparación.** Se comparan los valores X_n de forma exacta y los U_n con una tolerancia de 10^{-9}.

```
x = semilla
Para n = 1 hasta 10000:
    x = (a * x + c) mod m
    u = x / m
    Guardar (n, x, u)
Guardar los 10000 valores en un archivo CSV
```

5.3. Implementación computacional: pasos realizados

Estos fueron los pasos que realicé, en el orden que indica la ficha:

1. **Seleccioné los parámetros.** Usé $a = 1664525$, $c = 1013904223$, $m = 2^{32}$ y semilla 12345 (Tabla 1).
2. **Organicé el programa en modelo, vista y controlador.** Lo hice igual en Python y en Java, para que cada parte tuviera una sola tarea:
   - el *modelo* guarda la fórmula del generador (`generador_lcg.py` y `GeneradorLCG.java`) y los cálculos de análisis: media, varianza, valores repetidos y período (`analisis.py` y `Analisis.java`); en Python también compara con los datos de Java (`comparador.py`);
   - la *vista* muestra los resultados en la terminal, guarda el CSV y, en Python, dibuja las gráficas;
   - el *controlador* sigue los pasos de la ficha y le pasa los resultados a la vista.
3. **Generé 10 000 valores en Python y en Java.** Desde la carpeta del proyecto, el script `ejecutar.sh` compila y ejecuta Java y después Python. Cada lenguaje guarda un CSV por semilla en la carpeta `resultados` (por ejemplo, `lcg_python_12345.csv` y `lcg_java_12345.csv`). Los comandos son:

```
./ejecutar.sh
# o por separado (Java primero):
javac -encoding UTF-8 -d java/build $(find java/src -name "*.java")
java -cp java/build Main
python3 python/main.py
```

4. **Comparé las primeras 20 observaciones.** Python lee el CSV que generó Java y muestra una tabla con los dos valores de cada posición (Tabla 3). Después cuenta en cuántos de los 10 000 valores coinciden.
5. **Grafiqué la secuencia.** Con Matplotlib hice tres gráficas: los primeros 100 valores, un histograma de los 10 000 valores y un gráfico de U_n contra U_{n+1} (Tabla 2).
6. **Analicé el período.** Con $m = 16$ conté el período en tres casos de prueba para ver qué pasa con parámetros inadecuados. En Java también medí el período completo con $m = 2^{32}$, contando los pasos hasta volver a la semilla (se pide con `./ejecutar.sh periodo`). En Python no lo hice, porque con más de 4 mil millones de pasos tardaría horas.
7. **Cambié la semilla.** Repetí todo con la semilla 54321 para ver qué cambia en la secuencia y en sus estadísticas.
8. **Comparé los resultados.** Verifiqué que Python y Java coinciden con las dos semillas y comparé la media y la varianza de cada secuencia con los valores esperados para una distribución uniforme en $[0, 1)$: media 0,5 y varianza $1/12 \approx 0{,}0833$.

5.4. Gráficas del informe

Tabla 2: Figuras del informe, con el nombre del archivo de cada una.

Fig.	Archivo (en `figures/`)	Contenido
1	`secuencia.png`	Primeros 100 valores de U_n con las semillas 12345 y 54321.
2	`histograma.png`	Cantidad de valores en cada décimo de $[0, 1)$, con los 10 000 valores de la semilla 12345.
3	`dispersion.png`	Puntos (U_n, U_{n+1}) de los 10 000 valores de la semilla 12345.

6. Resultados esperados

Según la guía de la práctica, se espera [1]:

- Obtener dos implementaciones funcionales que produzcan la misma secuencia cuando utilizan los mismos parámetros.
- Armar una tabla con los datos generados por ambos programas e ir comparándolos.
- Dar las conclusiones del análisis.

Con el trabajo realizado logré obtener las dos implementaciones, las tablas de comparación, las gráficas, el análisis del período y la prueba con una segunda semilla.

7. Resultados

7.1. Primeras 20 observaciones

La Tabla 3 muestra los primeros 20 valores generados con la semilla 12345 en cada lenguaje. Coinciden en las 20 posiciones, tanto en X_n como en U_n.

Tabla 3: Primeras 20 observaciones con la semilla 12345, generadas por Python y por Java.

n	X_n Python	X_n Java	U_n Python	U_n Java	¿Igual?
1	87628868	87628868	0,0204026857	0,0204026857	Sí
2	71072467	71072467	0,0165478482	0,0165478482	Sí
3	2332836374	2332836374	0,5431557945	0,5431557945	Sí
4	2726892157	2726892157	0,6349040561	0,6349040561	Sí
5	3908547000	3908547000	0,9100295138	0,9100295138	Sí
6	483019191	483019191	0,1124616691	0,1124616691	Sí
7	2129828778	2129828778	0,4958894053	0,4958894053	Sí
8	2355140353	2355140353	0,5483488443	0,5483488443	Sí
9	2560230508	2560230508	0,5961001171	0,5961001171	Sí
10	3364893915	3364893915	0,7834504161	0,7834504161	Sí
11	171172990	171172990	0,0398543174	0,0398543174	Sí
12	3194601925	3194601925	0,7438012224	0,7438012224	Sí
13	4148119648	4148119648	0,9658093676	0,9658093676	Sí
14	316399679	316399679	0,0736675409	0,0736675409	Sí
15	3004788882	3004788882	0,6996069294	0,6996069294	Sí
16	1976948425	1976948425	0,4602941743	0,4602941743	Sí
17	1702883732	1702883732	0,3964835154	0,3964835154	Sí
18	4121112547	4121112547	0,9595212869	0,9595212869	Sí
19	1744294886	1744294886	0,4061253010	0,4061253010	Sí
20	4092090893	4092090893	0,9527641565	0,9527641565	Sí

7.2. Los 10 000 valores con dos semillas

Tabla 4: Comparación entre Python y Java y estadísticas de cada secuencia de 10 000 valores.

Semilla	Valores iguales (Python y Java)	Media	Varianza	X repetidos
12345	10 000 de 10 000	0,498770	0,083012	0
54321	10 000 de 10 000	0,498870	0,083588	0
Esperado (uniforme)	–	0,500000	0,083333	0

Tabla 5: Primeros cinco valores de cada semilla (iguales en Python y en Java).

n	X_n con semilla 12345	X_n con semilla 54321
1	87 628 868	1 238 253 532
2	71 072 467	1 708 513 675
3	2 332 836 374	3 683 344 750
4	2 726 892 157	1 863 498 229
5	3 908 547 000	2 137 457 360

7.3. Período

El período máximo posible es $m = 4\,294\,967\,296$. El programa Java contó los pasos hasta volver a la semilla y obtuvo **4 294 967 296 pasos** en 13,7 segundos, es decir, el período completo.

La Tabla 6 muestra el efecto de los parámetros en un caso pequeño ($m = 16$), donde el período se puede contar completo en los dos lenguajes.

Tabla 6: Período con $m = 16$ para tres combinaciones de parámetros (mismo resultado en Python y Java).

a	c	m	Semilla	Período	Comentario
5	3	16	1	16	Parámetros adecuados: período completo.
5	0	16	1	4	Con $c = 0$ el período se acorta.
1	0	16	3	1	Con $a = 1$ y $c = 0$ el valor nunca cambia.

7.4. Gráficas

Las Figuras 1 a 3 muestran la secuencia generada y sus propiedades.

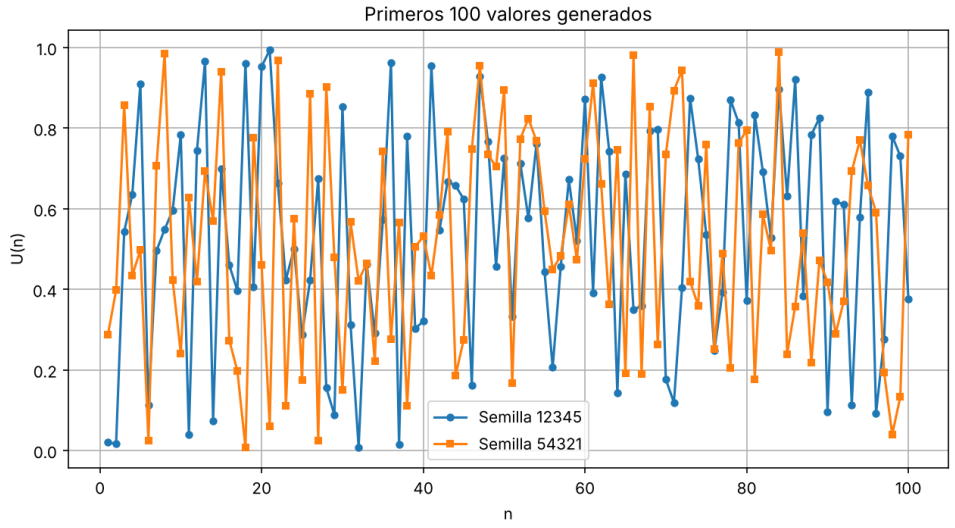


Figura 1: Primeros 100 valores de U_n con las semillas 12345 y 54321.

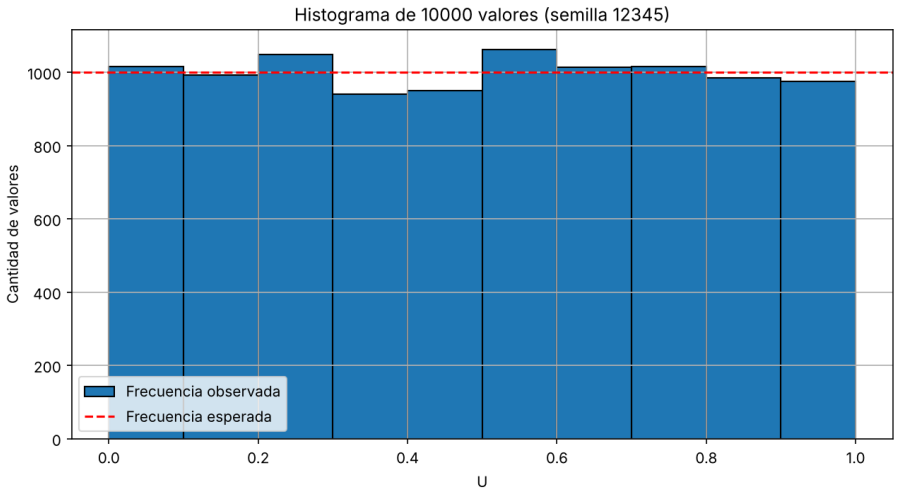


Figura 2: Histograma de los 10 000 valores de la semilla 12345. La línea roja es la frecuencia esperada (1000 por intervalo).

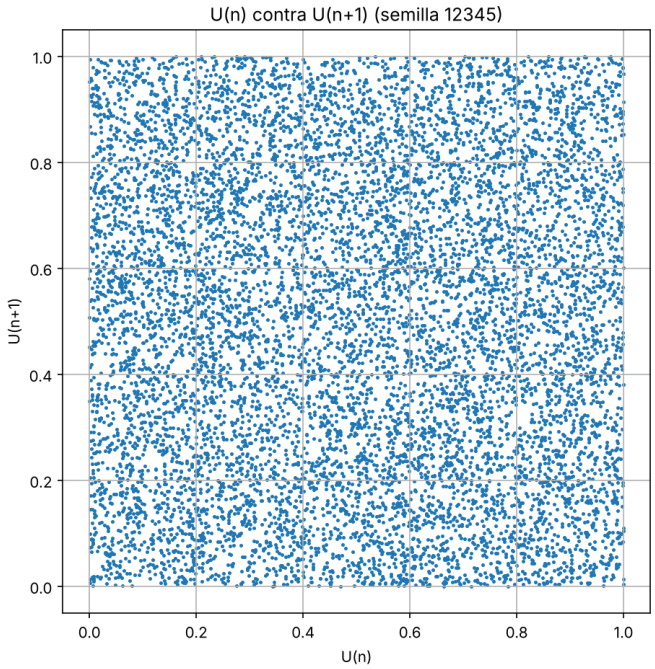


Figura 3: Puntos (U_n, U_{n+1}) de los 10 000 valores de la semilla 12345.

7.5. Comparación de las dos implementaciones

Tabla 7: Diferencias que encontré al programar el mismo generador en Python y en Java.

Aspecto	Python	Java
Enteros	Los enteros no tienen límite de tamaño, así que $a \cdot X$ nunca se desborda.	Se usa `long` (64 bits). El producto $a \cdot X$ llega a unos 7×10^{15} y cabe sin desbordarse.
Fórmula	`x = (a * x + c) % m`	`x = (a * x + c) % m;`

Aspecto	Python	Java
Guardar los valores	Lista de tuplas `(n, x, u)`.	Lista de objetos de la clase `Valor`.
Formato de decimales	El formato `.10f` escribe siempre el punto decimal.	Hay que indicar `Locale.US`; sin eso, un sistema en español escribiría comas y rompería el CSV.
Gráficas	Con Matplotlib, que se instala aparte.	La biblioteca estándar no trae gráficas, por eso se hacen en Python.
Resultado	Misma secuencia	Misma secuencia

8. Análisis de Resultados

8.1. Comparación entre Python y Java (Tablas 3 y 4)

Lo primero que revisé fue si los dos programas daban lo mismo. En la Tabla 3 las 20 primeras observaciones son idénticas, y al comparar los 10 000 valores, Python y Java coincidieron en todos, con las dos semillas. Esto tiene sentido porque X_n se calcula solo con enteros y con la misma fórmula, así que no hay redondeos que separen a los dos lenguajes. Como los U_n se obtienen dividiendo entre m, también coinciden. Con esto se cumple el resultado esperado de la guía: dos implementaciones que producen la misma secuencia con los mismos parámetros.

Aquí vale la pena notar una diferencia con la práctica anterior (modelo SIR), donde Java y NetLogo daban resultados distintos aunque usaran la misma semilla. En ese caso cada herramienta usaba su propio generador interno; en esta práctica yo programé el generador, y por eso sí puedo obtener exactamente los mismos números.

8.2. Efecto de cambiar la semilla (Figura 1, Tabla 5)

En la Figura 1 se ven los primeros 100 valores con las dos semillas. Las dos curvas suben y bajan entre 0 y 1 sin una tendencia clara, pero no coinciden en ningún punto: desde el primer valor son distintas (por ejemplo, X_1 es 87 628 868 con la semilla 12345 y 1 238 253 532 con la semilla 54321, Tabla 5). Es decir, la semilla decide toda la secuencia. Aun así, las estadísticas de la Tabla 4 son muy parecidas: la media está en 0,4988 en los dos casos y la varianza en 0,0830 y 0,0836. Cambiar la semilla cambia los números, pero no la calidad del generador.

8.3. Uniformidad (Figura 2, Tabla 4)

Si los valores son uniformes, cada décimo de $[0, 1)$ debería tener unos 1000 de los 10 000 valores. En la Figura 2 las barras quedan cerca de esa línea: la más baja tiene 940 valores (entre 0,3 y 0,4) y la más alta 1063 (entre 0,5 y 0,6). Son diferencias pequeñas, de hasta un 6 %, y es normal que aparezcan en una muestra de 10 000 números. La media (0,4988) y la varianza (0,0830) también están muy cerca de lo esperado para una uniforme (0,5 y 0,0833). Con eso puedo decir que los valores se reparten de forma bastante pareja, pero no es una prueba estadística formal: solo comparé la media, la varianza y el histograma.

8.4. Relación entre un valor y el siguiente (Figura 3)

En la Figura 3 cada punto es un par (U_n, U_{n+1}). Si un valor dependiera mucho del anterior, los puntos formarían líneas o zonas vacías. Lo que se ve es que los puntos llenan todo el cuadrado de forma pareja, sin un patrón evidente. Esto indica que, al menos entre valores consecutivos, no hay una relación visible. Hay que tener presente que el generador sí es una fórmula determinista: cada valor se calcula exactamente a partir del

anterior, y que no se vea un patrón en este gráfico no significa que no exista.

8.5. Período (Tabla 6)

Con los parámetros de la ficha el período es completo: 4 294 967 296 valores antes de repetirse, como lo midió Java al contar los pasos hasta volver a la semilla. Los 10 000 valores que generé son una parte muy pequeña de ese ciclo (alrededor del 0,0002 %), y por eso no hubo ningún X repetido en las dos secuencias.

La Tabla 6 muestra qué pasa cuando los parámetros no son buenos. Con $m = 16$, los valores $a = 5$ y $c = 3$ dan el período completo (16). Si $c = 0$, el período baja a 4, y con $a = 1$ y $c = 0$ el valor ni siquiera cambia: el período es 1. Esto me ayudó a entender por qué no se pueden elegir los parámetros al azar: un generador con parámetros inadecuados repite los mismos pocos números una y otra vez.

8.6. Limitaciones

Este trabajo tiene límites. Probé solo dos semillas, así que no puedo afirmar nada general sobre todas las semillas. Las propiedades que revisé (media, varianza, histograma y dispersión) son básicas y no reemplazan las pruebas estadísticas de uniformidad e independencia. Además, aunque el período es largo, un LCG con $m = 2^{32}$ no sirve para usos que exigen números impredecibles, como la criptografía. Python no mide el período completo porque tardaría horas; esa medición se hizo en Java.

9. Preguntas de Control

1. **¿Qué significa pseudoaleatorio?**

   Significa que los números parecen aleatorios, pero en realidad salen de una fórmula exacta [2]. Por eso, si se conocen los valores de entrada, la secuencia se puede predecir y repetir [2, 3].

2. **¿Qué función cumple la semilla?**

   La semilla es el valor inicial Z_0 con el que el generador arranca y calcula toda la secuencia [2, 4]. Sirve para obtener siempre los mismos resultados y para facilitar la revisión del código, ya que una misma semilla genera la misma serie [3, 4].

3. **¿Qué significa período?**

   Es la cantidad de números que el generador produce antes de empezar a repetir la misma secuencia [2]. Como los valores posibles son limitados, tarde o temprano el generador vuelve a un valor anterior [2].

4. **¿Qué ocurre si se utilizan parámetros inadecuados?**

   La secuencia se repite mucho antes de lo esperado y los números dejan de ser uniformes e independientes, apareciendo patrones que afectan la calidad de la simulación [2].

5. **¿Qué ventaja tiene implementar el mismo algoritmo en dos lenguajes?**

   Permite comprobar que el algoritmo está bien implementado y sin errores. Si ambos programas dan los mismos resultados, se confirma que el modelo es consistente, funciona en distintos entornos y se puede reproducir [5].

10. Conclusiones

- Implementé el generador congruencial lineal en Python y en Java, y con los mismos parámetros ($a = 1664525$, $c = 1013904223$, $m = 2^{32}$) y las semillas 12345 y 54321 las

dos versiones produjeron exactamente los mismos 10 000 valores. Esto se debe a que X_n se calcula solo con enteros, sin redondeos.

- La semilla determina por completo la secuencia: al cambiarla de 12345 a 54321 los valores fueron distintos desde el primero, pero la media (0,4988) y la varianza (0,083) se mantuvieron cerca de lo esperado para una distribución uniforme.
- Con los parámetros de la guía el período es completo (4 294 967 296), por lo que los 10 000 valores no tienen repeticiones. En las pruebas con $m = 16$ vi que con parámetros inadecuados el período se reduce mucho.
- Las gráficas muestran valores repartidos de forma pareja y sin un patrón visible entre un valor y el siguiente. Estas revisiones son básicas: no sustituyen las pruebas estadísticas formales, y solo se probaron dos semillas.

11. Declaración de uso de Inteligencia Artificial

En esta práctica utilicé dos herramientas de inteligencia artificial: Claude me sirvió para que me explicara el significado de las variables de las fórmulas del generador congruencial lineal y NotebookLM lo usé para apoyar la redacción de las preguntas de control, con base en los libros y documentos que cargué como fuentes.

11.1. Prompts utilizados

Prompt 1: Claude (explicación de las variables de las fórmulas).

> *Tengo la fórmula del generador congruencial lineal y no entiendo bien qué representa cada letra. Explícame con palabras sencillas qué significan Z_0, a, c, m y U_i, y para qué sirve cada una dentro de la fórmula.*

Prompt 2: NotebookLM (preguntas de control).

> *Usando únicamente las fuentes cargadas en este cuaderno, responde las siguientes preguntas de control: (1) ¿Qué significa pseudoaleatorio? (2) ¿Qué función cumple la semilla? (3) ¿Qué significa período? (4) ¿Qué ocurre si se utilizan parámetros inadecuados? (5) ¿Qué ventaja tiene implementar el mismo algoritmo en dos lenguajes? Redacta cada respuesta en un párrafo corto y con lenguaje sencillo. No agregues información que no esté en las fuentes. Al final de cada respuesta indica de qué fuente proviene y entrégame las referencias completas en formato IEEE.*

11.2. Justificación del uso

Herramientas utilizadas.

- **Claude:** explicación del significado de las variables de las fórmulas del generador.
- **NotebookLM:** apoyo para responder las preguntas de control a partir de las fuentes cargadas y obtener las citas en formato IEEE.

Trabajo realizado por el estudiante.

- Implementé y ejecuté los programas en Python y Java, y comprobé que ambos generan los mismos resultados.
- Revisé en las fuentes originales que lo propuesto por las herramientas fuera correcto, corregí el texto y verifiqué las referencias.

12. Bibliografía

[1] J. O. Guamán Q., “Guía de actividades práctico-experimentales nro. 004: Análisis comparativo de herramientas de simulación mediante un modelo común.” FEIRNNR, Carrera de Computación, Universidad Nacional de Loja. Documento proporcionado por el docente, 2026.

[2] J. S. Smith and D. T. Sturrock, *Simio and Simulation: Modeling, Analysis, Applications*. Sewickley, PA, USA: Simio LLC, 7th ed., August 2024.

[3] U. Wilensky, *The NetLogo 7.0.4 User Manual*. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL, USA, 2025.

[4] I. Romanowska, C. D. Wren, and S. A. Crabtree, *Agent-Based Modeling for Archaeology: Simulating the Complexity of Societies*. Santa Fe, NM, USA: The SFI Press, 2021.

[5] A. Heather, T. Monks, A. Harper, F. Alidoost, R. Challen, T. Slater, and N. Mustafee, "Reproducible analytical pipelines for healthcare discrete-event simulation: An open guide and worked examples," *NIHR Open Research*, vol. 6, pp. 1–19, June 2026.

13. Anexos

13.1. Repositorio del proyecto

https://github.com/C-ael/Simulacion_Portafolio